



Nama: Bintang Fikri Fauzan, M. Fakhri Nur, Rafki
Haykhal Alif
Mata Kuliah: Sistem/Teknologi Multimedia (IF25-40305)

Tugas: Final Project
Tanggal: 12 Desember 2025

1 Deskripsi Project

AirBeats – Touchless Piano Tiles dikembangkan sebagai permainan multimedia interaktif yang memanfaatkan deteksi *gesture* tangan secara *real-time* untuk menghasilkan pengalaman bermain tanpa sentuhan. Tujuan utama proyek ini adalah menyediakan mekanisme interaksi yang lebih natural dan higienis melalui pemetaan gerakan *tap* pada ruang udara ke dalam aksi menekan *tile* musik. Konsep *gesture detection* diterapkan dengan menangkap pergerakan jari melalui kamera dan menerjemahkannya menjadi input yang memicu suara piano serta evaluasi skor permainan. *Gameplay* berfokus pada ketepatan waktu pemain dalam menekan *tile* yang jatuh, sehingga unsur audio-visual bekerja secara sinkron untuk mendukung respons imersif. Dari perspektif multimedia, proyek ini relevan karena menggabungkan pemrosesan video, audio dinamis, serta umpan balik visual secara terintegrasi dalam satu sistem interaktif. Teknologi *touchless interaction* memberikan manfaat berupa pengalaman bermain yang lebih intuitif, responsif, dan sesuai dengan tren *contact-free interface* yang semakin banyak diadopsi dalam aplikasi modern.

2 Teknologi yang Digunakan

2.1 Python

Python adalah bahasa pemrograman tingkat tinggi yang dirancang untuk komputasi umum, sains data, dan aplikasi multimedia berkat sintaksnya yang sederhana serta ekosistem *library* yang luas. Dalam sistem multimedia, Python berperan sebagai pengelola alur logika, integrasi modul visual dan audio, serta platform utama pemrosesan *real-time*. Dalam AirBeats, Python digunakan untuk mengatur seluruh struktur permainan, termasuk deteksi *gesture*, manajemen *tile*, audio, *state machine*, dan *rendering* visual [1].

2.2 MediaPipe Hands

MediaPipe Hands merupakan model deteksi tangan *real-time* yang memetakan 21 titik *hand landmark* menggunakan pendekatan *machine learning* dan *graph-based processing pipeline*. Teknologi ini umum digunakan untuk *gesture recognition*, interaksi tanpa sentuhan, AR/VR, dan aplikasi visi komputer. Dalam AirBeats, MediaPipe menangkap posisi jari pemain dan memberikan koordinat akurat untuk menentukan aksi *tap* berdasarkan perubahan posisi vertikal jari [2].

2.3 OpenCV (cv2)

OpenCV adalah pustaka visi komputer *open-source* yang menyediakan fungsi untuk membaca video, memproses citra, dan melakukan transformasi visual secara efisien. Teknologi ini menjadi standar dalam aplikasi multimedia seperti pendeteksian objek, pemrosesan frame kamera, dan integrasi sistem *real-time*.

Pada AirBeats, OpenCV bertugas menangkap input kamera, mengirimkan frame ke MediaPipe, dan menyediakan *NumPy array* yang kemudian ditampilkan kembali melalui Pygame sebagai latar visual permainan [3].

2.4 NumPy

NumPy adalah pustaka komputasi numerik yang menyediakan struktur *array* multidimensi serta operasi matematika berkecepatan tinggi. NumPy menjadi fondasi bagi banyak sistem pemrosesan multimedia dan visi komputer. Dalam AirBeats, NumPy digunakan untuk menangani data frame dari OpenCV, perhitungan pergerakan jari antar frame, serta operasi matematis lain yang diperlukan oleh modul deteksi *gesture* [4].

2.5 Pygame

Pygame adalah pustaka multimedia berbasis Python untuk menangani grafis 2D, audio, *event handling*, dan *game loop* secara *real-time*. Dalam konteks multimedia, Pygame digunakan untuk mengembangkan aplikasi interaktif yang memerlukan sinkronisasi visual dan audio. AirBeats memanfaatkan Pygame untuk menampilkan *tile* jatuh, HUD skor, efek visual, serta memainkan suara piano setiap kali *gesture tap* terdeteksi [5].

3 Cara Kerja Program

- **Kamera Membaca Frame**

Proses dimulai ketika sistem menangkap frame secara kontinu melalui *webcam* menggunakan OpenCV. Setiap frame dibaca dalam format BGR kemudian dikonversi menjadi RGB agar kompatibel dengan MediaPipe. Frame ini juga dapat diperbesar atau disesuaikan ukurannya sebelum diproses lebih lanjut. Setelah itu, frame disimpan sebagai *surface* yang dapat ditampilkan kembali pada layar permainan melalui Pygame. Tahap ini menjadi fondasi bagi seluruh proses interaksi berbasis *gesture*.

- **MediaPipe Mendeteksi Tangan**

Setelah frame diperoleh, MediaPipe Hands melakukan analisis untuk mendeteksi posisi tangan dan menghasilkan 21 titik *landmark* untuk tiap tangan. Titik-titik ini mewakili sendi dan ujung jari, sehingga sistem dapat mengetahui posisi jari pemain secara presisi. Koordinat yang dihasilkan kemudian dinormalisasi ke dalam dimensi frame sehingga dapat dipakai dalam analisis temporal. Jika tangan tidak terdeteksi, sistem tetap melanjutkan pembacaan frame berikutnya tanpa memicu input. Tahap ini memastikan permainan selalu mendapatkan data *gesture* yang stabil dan akurat.

- **Deteksi Gesture Tap**

Deteksi *gesture* dilakukan dengan memantau perubahan posisi vertikal pada jari tertentu seperti jari telunjuk, tengah, manis, dan kelingking. Jika penurunan posisi jari melewati ambang batas tertentu, gerakan tersebut ditafsirkan sebagai *tap*. Pendekatan ini efektif karena gerakan *tap* umumnya diawali dengan pergerakan cepat ke bawah yang mudah diukur secara numerik. Proses *smoothing* diterapkan untuk mengurangi noise agar deteksi lebih konsisten pada berbagai kondisi. Hasil deteksi *gesture* kemudian diteruskan ke modul *gameplay* sebagai input *lane*.

- **Game Manager Menerima Input Lane**

Ketika *gesture tap* terdeteksi, sistem mengirimkan input berupa nomor *lane* ke **GameManager**. Modul ini memverifikasi apakah permainan sedang berada pada state yang menerima input, yaitu mode *gameplay* aktif. Selanjutnya, **GameManager** meneruskan input tersebut ke **TileManager** untuk memeriksa apakah ada *tile* yang berada di *hit zone*. Modul ini juga menangani pembaruan

statistik seperti jumlah tap, waktu permainan, dan kondisi transisi state. Dengan demikian, **GameManager** berfungsi sebagai penghubung antara gesture pemain dan logika inti permainan.

- **Tile System Menjatuhkan Tile dan Mengecek Hit/Miss**

Tile dijatuhkan berdasarkan waktu kemunculan yang ditentukan oleh lagu melalui **SongManager**. Setiap tile bergerak ke bawah dengan kecepatan sesuai tingkat kesulitan hingga mencapai area *hit zone*. Saat pemain melakukan tap, **TileManager** memeriksa apakah tile berada dalam zona tersebut dan menentukan kualitas hit seperti *PERFECT*, *GOOD*, atau *BAD*. Jika tile lewat tanpa terkena tap, tile dianggap miss dan dicatat sebagai kesalahan pemain. Sistem tile ini memastikan sinkronisasi antara musik, pergerakan tile, dan gesture pemain.

- **Audio Manager Memainkan Note**

Ketika tile berhasil di-hit, **AudioManager** memainkan nada piano yang sesuai dengan tile tersebut. Pemutaran audio dilakukan menggunakan **pygame.mixer** yang mampu menghasilkan suara dengan latensi rendah. Setiap nada disimpan sebagai file WAV sehingga respons audio menjadi cepat dan jernih saat dipicu oleh gesture pemain. Proses ini berjalan paralel dengan logika gameplay sehingga tidak mengganggu *frame rendering*. Dengan demikian, pemain mendapatkan pengalaman audio-visual yang sinkron dan responsif.

- **Skor, Combo, dan State Game Diperbarui**

Setelah sistem menentukan nilai hit, **ScoreManager** menambahkan poin sesuai kualitas pukulan pemain. Pada saat yang sama, **ComboCounter** meningkatkan atau mereset combo tergantung kondisi hit atau miss. Informasi ini kemudian ditampilkan pada HUD permainan agar pemain dapat memantau performa mereka secara *real-time*. Sementara itu, **GameManager** mengevaluasi apakah kondisi tertentu telah tercapai, seperti *game over* akibat terlalu banyak miss. Tahap ini memastikan permainan berjalan dinamis dan evaluatif hingga lagu berakhir.

4 Penjelasan Program

4.1 main.py

File **main.py** berfungsi sebagai titik masuk utama sistem dan menyatukan seluruh pipeline multimedia: pembacaan kamera, deteksi tangan melalui MediaPipe, interpretasi gesture *tap*, serta pengiriman input lane menuju **GameManager**. File ini juga bertugas membuat window Pygame, menginisialisasi tracking variabel jari, dan menjalankan game loop yang mengatur pembaruan logika serta *rendering*. Seluruh interaksi antara kamera, MediaPipe, dan subsistem permainan dimulai dari file ini sehingga perannya sangat sentral dalam keseluruhan alur kerja AirBeats.

4.1.1 Kode program main

```

1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import pygame
5 import sys
6
7 sys.path.insert(0, 'src')
8
9 from game_manager import GameManager
10
11 def main():
12     pygame.init()
13
14     mp_hands = mp.solutions.hands
15     mp_drawing = mp.solutions.drawing_utils

```

```

16 cap = cv2.VideoCapture(0)
17 if not cap.isOpened():
18     print("Cannot open camera")
19     return
20
21
22 cam_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
23 cam_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
24
25 window_width = 1280
26 window_height = 720
27
28 screen = pygame.display.set_mode((window_width, window_height))
29 pygame.display.set_caption("AirBeats - Touchless Piano Tiles")
30 clock = pygame.time.Clock()
31
32 game_manager = GameManager(window_width=window_width, window_height=window_height)
33
34 prev_y = {"index": None, "middle": None, "ring": None, "pinky": None}
35 THRESHOLD = 20
36
37 finger_to_lane = {
38     "index": 0,
39     "middle": 1,
40     "ring": 2,
41     "pinky": 3
42 }
43
44 print("\nAirBeats - Touchless Piano Tiles")
45 print("Controls:")
46 print(" [MOUSE] Navigate Menu")
47 print(" [ESC]   Back / Pause")
48
49 running = True
50 with mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7) as hands:
51     while running:
52         ret, frame = cap.read()
53         if not ret:
54             break
55
56         frame = cv2.flip(frame, 1)
57
58         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
59         result = hands.process(rgb)
60
61         if result.multi_hand_landmarks:
62             for handLms in result.multi_hand_landmarks:
63                 mp_drawing.draw_landmarks(frame, handLms, mp_hands.HAND_CONNECTIONS)
64
65                 h, w, _ = frame.shape
66                 landmarks = handLms.landmark
67
68                 fingers = {
69                     "index": landmarks[8],
70                     "middle": landmarks[12],
71                     "ring": landmarks[16],
72                     "pinky": landmarks[20]
73                 }
74
75                 coords = {}
76                 for name, lm in fingers.items():
77                     coords[name] = (int(lm.x * w), int(lm.y * h))
78

```

```

79         for name, (x, y) in coords.items():
80             if prev_y[name] is not None:
81                 y_smoothed = int(0.7 * prev_y[name] + 0.3 * y)
82                 diff = prev_y[name] - y_smoothed
83
84                 if diff < -THRESHOLD:
85                     # Visual feedback on OpenCV frame
86                     cv2.circle(frame, (x, y), 20, (0, 255, 255), cv2.FILLED)
87                     cv2.putText(frame, "TAP!", (x-30, y-30),
88                                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,255), 2)
89
90                     lane_id = finger_to_lane.get(name)
91                     if lane_id is not None:
92                         game_manager.handle_input(lane_id)
93
94                     prev_y[name] = y_smoothed
95             else:
96                 prev_y[name] = y
97
98                 cv2.circle(frame, (x, y), 10, (0, 255, 0), cv2.FILLED)
99
100         frame_resized = cv2.resize(frame, (window_width, window_height))
101
102         frame_rgb = cv2.cvtColor(frame_resized, cv2.COLOR_BGR2RGB)
103         frame_surface = pygame.image.frombuffer(frame_rgb.tobytes(), frame_rgb.shape
104 [1::-1], "RGB")
105
106         game_manager.current_frame_surface = frame_surface
107
108         for event in pygame.event.get():
109             if event.type == pygame.QUIT:
110                 running = False
111
112             game_manager.handle_event(event)
113
114             if game_manager.should_exit:
115                 running = False
116
117         game_manager.update()
118         game_manager.draw(screen)
119         pygame.display.flip()
120         clock.tick(60)
121
122         cap.release()
123         pygame.quit()
124         game_manager.cleanup()
125         sys.exit()
126
127 if __name__ == "__main__":
128     main()

```

Fungsi-fungsi penting:

- **main()** – Memulai seluruh sistem: inisialisasi Pygame, perangkat kamera, MediaPipe Hands, dan instance **GameManager**.
- **Loop utama** – Membaca frame kamera secara kontinu, memproses landmark tangan, mendeteksi event *tap*, mengirim input lane ke **GameManager**, memperbarui status permainan, dan merender tampilan.

4.1.2 Alur kerja dan logika file

Alur program diawali dengan inisialisasi kamera dan setup komponen (Pygame, MediaPipe, GameManager). Setelah itu program memasuki `while`-loop utama yang terus menangkap frame; tiap frame dikonversi ke format yang sesuai dan diteruskan ke MediaPipe untuk memperoleh koordinat landmark jari. Perubahan posisi jari antar-frame diukur untuk mendeteksi gesture *tap*; bila terdeteksi, sistem memetakan lokasi jari ke nomor lane dan memanggil `GameManager.handle_input(lane)`. Selanjutnya `GameManager` menjalankan logika update, dan Pygame memperbarui layar sampai program dihentikan.

File ini berperan sebagai jembatan utama antara dunia nyata dengan dunia permainan. Tanpa `main.py`, aliran data *landmark* dan pengaktifan mekanisme interaksi tanpa sentuhan tidak akan pernah mencapai subsistem permainan sehingga game tidak dapat berfungsi.

4.2 game_manager.py

File `game_manager.py` merupakan pusat logika permainan yang mengatur state permainan, manajemen skor, *tile generation*, audio, efek visual, serta perpindahan antar layar antarmuka. File ini mengintegrasikan subsistem seperti `TileManager`, `AudioManager`, `SongManager`, `DifficultyManager`, dan `ScoreManager` sehingga gameplay dapat berjalan sinkron dan responsif.

Fungsi-fungsi penting:

4.2.1 start_game()

```

1 def start_game(self):
2     if not self.state_machine.transition_to(GameState.PLAYING):
3         return False
4
5     self.score_manager.reset()
6     self.timer.reset()
7
8     song_tiles = self.song_manager.get_current_song_tiles()
9     song_duration = self.song_manager.get_song_duration()
10
11     difficulty_speed = self.difficulty_manager.get_tile_speed()
12
13     current_song_obj = self.song_manager.get_song(self.song_manager.current_song)
14     speed_multiplier = getattr(current_song_obj, 'speed_multiplier', 1.0)
15     final_speed = difficulty_speed * speed_multiplier
16
17     self.tile_manager = TileManager(
18         self.window_width,
19         self.window_height,
20         speed=final_speed
21     )
22     self.tile_manager.load_song_tiles(song_tiles)
23     self.tile_manager.start_song()
24     self.timer.duration = song_duration
25     self.timer.mode = Timer.COUNTDOWN
26     self.timer.start()
27     self.game_session_active = True
28     current_song_id = self.song_manager.current_song
29     self.audio_manager.play_music(current_song_id)
30     self.screens[GameState.MENU].on_exit()
31     self.screens[GameState.PLAYING].on_enter()
32     self.current_screen = GameState.PLAYING
33
34     current_song = self.song_manager.songs[self.song_manager.current_song].name
35     current_diff = self.difficulty_manager.current_difficulty

```

```

36     print(f"GAME STARTED!")
37     print(f"Song: {current_song}")
38     print(f"Difficulty: {current_diff}")
39     print(f"Duration: {song_duration:.1f}s")
40     return True

```

start_game(): Fungsi untuk memulai sesi permainan: mereset skor, mengatur durasi lagu, memuat urutan tile, dan memulai pemutaran musik latar.

4.2.2 pause_game(), resume_game()

```

1  def pause_game(self):
2      if not self.state_machine.transition_to(GameState.PAUSED):
3          return False
4
5      self.timer.pause()
6      self.audio_manager.stop_music()
7      pygame.mixer.music.pause()
8
9      self.screens[GameState.PLAYING].on_exit()
10     self.screens[GameState.PAUSED].on_enter()
11     self.current_screen = GameState.PAUSED
12     print("GAME PAUSED!")
13
14     return True
15
16 def resume_game(self):
17     if not self.state_machine.transition_to(GameState.PLAYING):
18         return False
19
20     self.timer.start()
21     pygame.mixer.music.unpause()
22
23     self.screens[GameState.PAUSED].on_exit()
24     self.screens[GameState.PLAYING].on_enter()
25     self.current_screen = GameState.PLAYING
26     print("GAME RESUMED!")
27
28     return True

```

pause_game(), resume_game(): Merupakan fungsi untuk mengatur mekanisme jeda dan melanjutkan permainan.

4.2.3 end_game()

```

1  def end_game(self):
2      if not self.state_machine.transition_to(GameState.GAME_OVER):
3          return False
4
5      self.timer.stop()
6      self.game_session_active = False
7      self.audio_manager.stop_music()
8
9      self.final_score = self.score_manager.total_score
10     self.final_max_combo = self.score_manager.max_combo
11
12     if hasattr(self, 'current_screen'):
13         self.current_screen = GameState.GAME_OVER
14
15     game_over_screen = self.screens[GameState.GAME_OVER]
16     game_over_screen.final_stats = self._get_game_stats()

```

```

17     game_over_screen.rank_data = game_over_screen._calculate_rank()
18
19     self.screens[GameState.PLAYING].on_exit()
20     self.screens[GameState.GAME_OVER].on_enter()
21
22     print("GAME OVER!")
23     print(f"Final Score: {self.final_score}")
24     return True

```

end_game(): Fungsi untuk mengakhiri sesi, menyimpan statistik akhir, serta memunculkan layar *Game Over*.

4.2.4 handle_input()

```

1 def handle_input(self, lane):
2     if not self.state_machine.is_playing():
3         return
4
5     hit_tile, quality = self.tile_manager.check_hit(lane)
6     if hit_tile:
7         self.on_tile_hit(hit_tile, quality)
8         # Play sound based on tile's note
9         self.audio_manager.play_note(hit_tile.note)
10    else:
11        pass

```

handle_input(): Fungsi untuk menerima input tap dari `main.py` dan mengecek hit/miss melalui `TileManager`

4.2.5 update()

```

1 def update(self):
2
3     if hasattr(self, 'current_screen') and self.current_screen in self.screens:
4         self.screens[self.current_screen].update()
5     else:
6         current_state = self.state_machine.current_state
7         if current_state in self.screens:
8             self.screens[current_state].update()
9
10    if self.state_machine.is_playing():
11        # Update Timer
12        if self.timer.is_time_up():
13            # Only end game if no more tiles are active or pending
14            if not self.tile_manager.tiles and not self.tile_manager.song_tiles:
15                self.end_game()
16                return
17
18        self.tile_manager.update()
19        self.vfx_manager.update()
20
21        missed_tiles = self.tile_manager.check_misses()
22        for tile in missed_tiles:
23            self.on_tile_miss()

```

update(): Fungsi untuk menjalankan pembaruan tile, efek visual, timer, serta mendeteksi akhir lagu

4.2.6 draw()


```

1 def draw(self, surface):
2     if hasattr(self, 'current_screen') and self.current_screen in self.screens:
3         self.screens[self.current_screen].draw(surface)
4     else:
5         current_state = self.state_machine.current_state
6         if current_state in self.screens:
7             self.screens[current_state].draw(surface)
8
9     if self.state_machine.is_playing():
10        self.vfx_manager.draw(surface)

```

draw(surface): Fungsi untuk menggambar tampilan permainan sesuai state (menu, playing, paused, game over)

4.3 timer.py

File `timer.py` menyediakan kelas utilitas `Timer` yang berfungsi untuk melacak waktu dalam permainan. Kelas ini mendukung dua mode operasi: **STOPWATCH** dan **COUNTDOWN**. Modul ini sangat penting untuk mengelola durasi lagu, menghitung sisa waktu permainan, serta menangani mekanisme jeda (*pause*) dan resume secara akurat tanpa kehilangan jejak waktu yang telah berlalu.

Fungsi-fungsi penting:

4.3.1 start()

```

1 def start(self):
2     if self.is_running and not self.is_paused:
3         print("Timer sudah berjalan!")
4         return False
5
6     if self.is_paused:
7         self.pause_time = None
8         self.is_paused = False
9         print("Timer resumed")
10        return True
11
12    self.start_time = time.time()
13    self.total_paused = 0
14    self.is_running = True
15    self.is_finished = False
16
17    print(f"Timer started ({self.mode})")
18    return True

```

start(): Memulai timer atau melanjutkan (*resume*) jika sedang dalam kondisi pause. Jika timer baru dimulai, fungsi ini mencatat waktu awal.

4.3.2 pause()

```

1 def pause(self):
2     if not self.is_running or self.is_paused:
3         print("Tidak bisa pause - timer tidak berjalan")
4         return False
5
6     self.pause_time = time.time()
7     self.is_paused = True
8
9     elapsed = self.get_elapsed_time()

```

```

10 print(f"Timer paused at {elapsed:.1f}s")
11 return True

```

pause(): Menghentikan sementara perhitungan waktu dengan mencatat *timestamp* saat jeda agar durasi pause tidak terhitung sebagai waktu bermain.

4.3.3 get_elapsed_time()

```

1 def get_elapsed_time(self):
2     if not self.is_running:
3         return 0.0
4
5     current_time = time.time()
6
7     if self.is_paused:
8         current_time = self.pause_time
9
10    elapsed = (current_time - self.start_time) - self.total_paused
11
12    return max(0.0, elapsed)

```

get_elapsed_time(): Menghitung total waktu berjalan dikurangi durasi pause. Ini adalah logika inti untuk mendapatkan waktu efektif permainan.

4.3.4 get_remaining_time()

```

1 def get_remaining_time(self):
2     if self.mode != self.COUNTDOWN:
3         return None
4
5     elapsed = self.get_elapsed_time()
6     remaining = self.duration - elapsed
7
8     return max(0.0, remaining)

```

get_remaining_time(): Menghitung sisa waktu permainan (khusus mode COUNTDOWN) dengan mengurangi durasi total dengan waktu yang telah berlalu.

4.3.5 get_display_time()

```

1 def get_display_time(self):
2     if self.mode == self.COUNTDOWN:
3         return self.format_time(self.get_remaining_time())
4     else:
5         return self.format_time(self.get_elapsed_time())

```

get_display_time(): Mengembalikan string waktu terformat (MM:SS) yang siap ditampilkan di UI, menyesuaikan dengan mode timer yang aktif.

4.4 game_state_machine.py

File `game_state_machine.py` berfungsi sebagai pengelola status aplikasi (*state management*) yang mendefinisikan berbagai kondisi permainan seperti **MENU**, **PLAYING**, **PAUSED**, dan **GAME_OVER**. Modul ini bertanggung jawab untuk memvalidasi dan mengeksekusi perpindahan antar state, memastikan bahwa alur permainan berjalan logis dan mencegah transisi yang tidak valid (misalnya, dari Menu langsung ke Game Over).

Fungsi-fungsi penting:**4.4.1 transition_to()**

```

1 def transition_to(self, new_state):
2     all_valid_states = [self.MENU, self.PLAYING, self.PAUSED, self.GAME_OVER]
3
4     if new_state not in all_valid_states:
5         return False
6
7     allowed_transitions = self.VALID_TRANSITIONS.get(self.current_state, [])
8
9     if new_state not in allowed_transitions:
10        print(f"ERROR: Cannot transition from {self.current_state} to {new_state}")
11        return False
12
13    self.previous_state = self.current_state
14    self.current_state = new_state
15
16    print(f"Transition: {self.previous_state} -> {self.current_state}")
17    return True

```

transition_to(new_state): Fungsi inti yang memvalidasi apakah state tujuan valid dan apakah transisi dari state saat ini ke state tujuan diperbolehkan. Jika valid, state akan diperbarui.

4.4.2 is_playing(), is_paused()

```

1 def is_playing(self):
2     return self.current_state == self.PLAYING
3 def is_paused(self):
4     return self.current_state == self.PAUSED

```

is_playing(), is_paused(): Fungsi *helper* boolean untuk memudahkan pengecekan status saat ini di modul lain tanpa perlu membandingkan string secara manual.

4.4.3 reset()

```

1 def reset(self):
2     self.previous_state = self.current_state
3     self.current_state = self.MENU
4     print(f"State reset to {self.MENU}")

```

reset(): Mengembalikan status permainan ke kondisi awal (MENU), biasanya dipanggil saat aplikasi baru dimulai atau di-restart total.

4.5 score_manager.py

File `score_manager.py` bertanggung jawab untuk mengelola sistem penilaian, pelacakan combo, dan statistik akurasi pemain. Modul ini menghitung skor berdasarkan ketepatan waktu ketukan dan multiplier combo, serta memantau jumlah *miss* untuk menentukan kondisi *Game Over*. Selain itu, `ScoreManager` juga menyimpan riwayat performa pemain selama sesi permainan berlangsung.

Fungsi-fungsi penting:

4.5.1 add_hit()

```
1 def add_hit(self, base_points=10):
2     multiplier = 1.0 + (self.current_combo * 0.1)
3
4     hit_score = int(base_points * multiplier)
5
6     self.total_score += hit_score
7     self.current_combo += 1
8     self.total_hits += 1
9
10    if self.current_combo > self.max_combo:
11        self.max_combo = self.current_combo
12
13    return hit_score
```

add_hit(): Menambahkan skor saat pemain berhasil menekan tile. Poin dihitung dengan mengalikan poin dasar dengan multiplier combo saat ini, lalu memperbarui status combo dan total hits.

4.5.2 add_miss()

```
1 def add_miss(self):
2     self.miss_count += 1
3
4     self.current_combo = 0
5
6     is_game_over = self.miss_count >= self.max_misses
7
8     return is_game_over
```

add_miss(): Menangani kejadian saat pemain melewati tile. Fungsi ini mereset combo ke nol, menambah penghitung *miss*, dan mengembalikan status *true* jika batas maksimal *miss* telah terlampaui (memicu Game Over).

4.5.3 get_accuracy()

```
1 def get_accuracy(self):
2     total_actions = self.total_hits + self.miss_count
3     if total_actions > 0:
4         return (self.total_hits / total_actions) * 100
5     return 0.0
```

get_accuracy(): Menghitung persentase akurasi permainan berdasarkan perbandingan antara total hits yang berhasil dengan total aksi (hits + misses).

4.5.4 reset()

```
1 def reset(self):
2     self.total_score = 0
3     self.current_combo = 0
4     self.max_combo = 0
5     self.miss_count = 0
6     self.total_hits = 0
7     self.combo_history = []
8     print("Score Manager reset for new game")
```

reset(): Mengembalikan semua statistik permainan (skor, combo, miss count) ke nol untuk memulai sesi permainan baru yang bersih.

4.6 tile_manager.py

File `tile_manager.py` bertanggung jawab atas pembuatan, pergerakan, dan rendering tile (balok nada) dalam permainan. Modul ini mengatur *spawning* tile berdasarkan data lagu, mendeteksi tabrakan (*collision*) untuk menentukan apakah pemain berhasil menekan tile di zona yang tepat, serta menghitung akurasi ketukan (*Perfect, Good, Bad*).

Fungsi-fungsi penting:

4.6.1 load_song_tiles()

```
1 def load_song_tiles(self, tiles_data):
2     self.song_tiles = tiles_data.copy()
3     self.song_mode = True
4     self.game_start_time = None
5     print(f"Loaded {len(self.song_tiles)} tiles")
```

load_song_tiles(): Memuat data urutan tile dari konfigurasi lagu ke dalam buffer internal `TileManager`, mempersiapkan sistem untuk mode permainan berbasis lagu.

4.6.2 update()

```
1 def update(self):
2     if self.song_mode and self.game_start_time:
3         current_time = time.time() - self.game_start_time
4
5         for tile_data in self.song_tiles[:]:
6             travel_distance = self.hit_zone_y + tile_height
7             pixels_per_second = self.speed * 60
8             travel_time_seconds = travel_distance / pixels_per_second
9             spawn_time = tile_data['time'] - travel_time_seconds
10
11            if current_time >= spawn_time - 0.1:
12                new_tile = Tile(...)
13
14                self.tiles.append(new_tile)
15                self.song_tiles.remove(tile_data)
16
17            for tile in self.tiles:
18                tile.update()
```

update(): Fungsi inti yang berjalan setiap frame. Ia menghitung kapan harus memunculkan tile baru agar sampai di garis target tepat sesuai irama lagu, serta memperbarui posisi vertikal semua tile yang sedang aktif.

4.6.3 check_hit()

```
1 def check_hit(self, lane):
2     for tile in self.tiles:
3         if tile.lane == lane and not tile.is_hit:
4             if (tile_bottom > self.hit_zone_y and
5                 tile_top < self.hit_zone_y + self.hit_zone_height):
6
7                 diff = abs(target_y - tile_bottom)
8                 quality = "BAD"
9                 if diff < 30: quality = "PERFECT"
10                elif diff < 60: quality = "GOOD"
```

```

12         tile.is_hit = True
13         tile.active = False
14         return tile, quality
15     return None, None

```

check_hit(): Memvalidasi input pemain pada jalur (*lane*) tertentu. Jika ada tile di zona hit saat input diterima, fungsi ini menghitung seberapa akurat ketukan tersebut dan mengembalikan kualitas penilaiannya.

4.6.4 check_misses()

```

1 def check_misses(self):
2     missed = []
3     for tile in self.tiles:
4         if not tile.is_hit and tile.active:
5             if tile.y > self.hit_zone_y + self.hit_zone_height:
6                 tile.active = False
7                 missed.append(tile)
8     return missed

```

check_misses(): Mendeteksi tile yang melewati batas bawah layar tanpa ditekan oleh pemain, yang kemudian akan dilaporkan sebagai *Miss* ke **ScoreManager**.

4.7 difficulty_manager.py

File **difficulty_manager.py** menyimpan preset parameter untuk setiap tingkat kesulitan (EASY sampai EXPERT). Parameter tersebut meliputi *tile_speed*, *spawn_rate*, dan *hit_window*, yang dapat diakses oleh **TileManager** dan **SongManager**. Modul ini memungkinkan pergantian tingkat kesulitan secara dinamis di dalam permainan.

Fungsi-fungsi penting:

4.7.1 set_difficulty()

```

1 def set_difficulty(self, difficulty_name):
2     if difficulty_name not in self.difficulties:
3         return False
4     self.current_difficulty = difficulty_name
5     return True

```

set_difficulty(name): Mengubah preset difficulty yang aktif.

4.7.2 get_tile_speed()

```

1 def get_tile_speed(self):
2     return self.difficulties[self.current_difficulty]['tile_speed']

```

get_tile_speed(): Mengembalikan kecepatan tile sesuai mode.

4.8 audio_manager.py

File **audio_manager.py** menginisialisasi mixer Pygame dengan konfigurasi latensi rendah, memuat sampel nada piano, memutar note ketika terjadi hit, serta memutar *background music* (BGM) untuk lagu yang sedang dimainkan. Modul ini memastikan sistem audio berjalan responsif dan selaras dengan ritme permainan.

Fungsi-fungsi penting:**4.8.1 load_notes()**

```

1 def load_notes(self):
2     notes_path = "assets/sounds/piano"
3     note_names = ['C', 'D', 'E', 'F', 'G', 'A', 'B', 'C_high']
4
5     for note in note_names:
6         filepath = os.path.join(notes_path, f"{note}.wav")
7         if os.path.exists(filepath):
8             self.notes[note] = pygame.mixer.Sound(filepath)
9             self.notes[note].set_volume(self.notes_volume)

```

load_notes(): Memuat file audio (WAV) untuk setiap nada piano ke dalam memori dan mengatur volume awalnya, memastikan aset siap diputar tanpa jeda saat permainan berlangsung.

4.8.2 play_note()

```

1 def play_note(self, note_name):
2     if note_name in self.notes:
3         self.notes[note_name].stop()
4         self.notes[note_name].play(maxtime=2000)

```

play_note(note_name): Memainkan efek suara nada piano yang sesuai dengan tile yang ditekan. Fungsi ini menghentikan instans suara sebelumnya dari nada yang sama untuk mencegah tumpang tindih audio yang berisik.

4.8.3 play_music(), stop_music()

```

1 def play_music(self, song_name):
2     music_path = f"assets/sounds/music/{song_name}.mp3"
3     if os.path.exists(music_path):
4         pygame.mixer.music.load(music_path)
5         pygame.mixer.music.play(-1) # Loop indefinitely
6         pygame.mixer.music.set_volume(0.5)
7 def stop_music(self):
8     pygame.mixer.music.stop()

```

4.9 songs.py

File **songs.py** mendefinisikan objek **Song** yang berisi sequence not dan beat, serta menyediakan fungsi konversi beat ke waktu. Selain itu tersedia **SongManager** yang mengelola daftar lagu, pemilihan lagu, dan pembuatan tile berdasarkan BPM dan multiplier kesulitan.

Fungsi-fungsi penting:**4.9.1 get_notes_with_timing()**

```

1 def get_notes_with_timing(self, bpm_multiplier=1.0):
2     adjusted_bpm = self.bpm * bpm_multiplier
3     seconds_per_beat = 60.0 / adjusted_bpm
4
5     tiles = []
6     previous_lane = -1
7

```

```

8   for note, beat in self.notes_sequence:
9       time = beat * seconds_per_beat
10
11      # Lane randomization logic
12      available_lanes = [0, 1, 2, 3]
13      if previous_lane != -1:
14          available_lanes.remove(previous_lane)
15          lane = random.choice(available_lanes)
16          previous_lane = lane
17
18      tiles.append({
19          'note': note,
20          'time': time,
21          'lane': lane
22      })
23
24      return tiles

```

4.9.2 get_current_song_tiles()

```

1  def get_current_song_tiles(self):
2      song = self.songs[self.current_song]
3      bpm_multiplier = self.difficulty_bpm_multipliers[self.current_difficulty]
4
5      tiles = song.get_notes_with_timing(bpm_multiplier)
6
7      print(f"Generated {len(tiles)} tiles for {song.name} ({self.current_difficulty})")
8      return tiles

```

4.9.3 get_song_duration()

```

1  def get_song_duration(self):
2      tiles = self.get_current_song_tiles()
3      if tiles:
4          return tiles[-1]['time'] + 2.0
5      return 60.0

```

4.10 vfx_manager.py

File `vfx_manager.py` menangani semua efek visual (*Visual Effects*) dalam permainan, termasuk partikel ledakan saat tile ditekan dan teks *popup* untuk umpan balik skor (*Perfect*, *Good*, *Miss*). Modul ini bertujuan untuk meningkatkan *game feel* dengan memberikan respons visual yang dinamis terhadap aksi pemain.

Fungsi-fungsi penting:

4.10.1 create_explosion()

```

1  def create_explosion(self, x, y, color):
2      for _ in range(20):
3          angle = random.uniform(0, 2 * math.pi)
4          speed = random.uniform(2, 10)
5          vx = math.cos(angle) * speed
6          vy = math.sin(angle) * speed
7
8          self.particles.append(Particle(

```



```

9         x, y, color, (vx, vy), life=random.uniform(0.5, 1.0), size=random.randint(4,
10         10)
        ))

```

4.10.2 create_score_popup()

```

1 def create_score_popup(self, x, y, text, color):
2     self.popups.append(ScorePopup(x, y, text, color))

```

4.10.3 update()

```

1 def update(self):
2     for p in self.particles:
3         p.update()
4     for popup in self.popups:
5         popup.update()
6
7     self.particles = [p for p in self.particles if p.life > 0]
8     self.popups = [p for p in self.popups if p.life > 0]

```

4.11 UI

Dalam pengembangan project AirBeats ini, disusun beberapa UI untuk screen dengan tampilan yang berbeda-beda. Seperti Menu, Settings, Pause, Game.

4.11.1 base_screen

```

1 # Initialization
2 def __init__(self, width, height, game_manager=None): ...
3
4 # Event Handling
5 def handle_event(self, event): ...
6
7 # Update Loop
8 def update(self): ...
9
10 # Rendering
11 def draw(self, surface): ...
12
13 # Screen Transition
14 def get_next_screen(self): ...
15 def set_next_screen(self, screen_name): ...
16
17 # State Management
18 def activate(self): ...
19 def deactivate(self): ...
20 def on_enter(self): ...
21 def on_exit(self): ...
22
23 # Utility
24 def get_screen_name(self): ...
25 def get_dimensions(self): ...
26 def get_center(self): ...
27 def __str__(self): ...

```

`base_screen` berfungsi sebagai kelas dasar yang menyediakan struktur standar untuk setiap layar dalam game, seperti menu, gameplay, pause, atau settings. Kelas ini menangani hal-hal fundamental

seperti penyimpanan ukuran layar, pengelolaan state aktif/nonaktif, mekanisme transisi ke screen berikutnya, serta menyediakan template untuk tiga fungsi utama game loop & event handling, update logic, dan rendering. Sehingga screen lain cukup melakukan override tanpa menulis ulang kerangka dasarnya. Dengan demikian, `base_screen` menjaga konsistensi, meminimalkan duplikasi kode, dan mempermudah pengembangan maupun penambahan screen baru dalam sistem game.

4.11.2 menu_screen

```

1 # Initialization
2 def __init__(self, width, height, game_manager=None): ...
3
4 # Event Handling
5 def handle_event(self, event): ...
6
7 # Button Navigation
8 def _next_button(self): ...
9 def _previous_button(self): ...
10 def _click_selected_button(self): ...
11
12 # Button Callbacks
13 def on_start_clicked(self): ...
14 def on_settings_clicked(self): ...
15 def on_exit_clicked(self): ...
16 def _get_transition_for_button(self, button_name): ...
17
18 # Update Loop
19 def update(self): ...
20
21 # Rendering
22 def draw(self, surface): ...
23
24 # Lifecycle
25 def on_enter(self): ...
26 def on_exit(self): ...
27
28 # Status
29 def is_exiting(self): ...
30 def get_selected_button(self): ...
31 def __str__(self): ...

```

`menu_screen` berfungsi sebagai layar utama yang menjadi titik awal interaksi pemain sebelum memasuki permainan. Screen ini menampilkan judul game, tombol navigasi seperti Start, Settings, dan Exit, serta menangani input keyboard dan mouse untuk berpindah antar opsi. Dengan mekanisme fokus tombol, navigasi, dan callback yang terstruktur, `menu_screen` menyediakan antarmuka awal yang intuitif sehingga pemain dapat memilih aksi berikutnya dengan mudah, sekaligus memanfaatkan sistem transisi dari `base_screen` untuk berpindah ke screen lain.

4.11.3 game_screen

```

1 # Initialization
2 def __init__(self, width, height, game_manager=None): ...
3
4 # Event Handling
5 def handle_event(self, event): ...
6
7 # Update Loop
8 def update(self): ...
9
10 # Rendering

```

```

11 def draw(self, surface): ...
12 def _draw_hud(self, surface): ...
13
14 # Lifecycle
15 def on_enter(self): ...
16 def on_exit(self): ...

```

game_screen adalah layar inti tempat gameplay berlangsung, yang menampilkan kamera atau background permainan, tile yang perlu ditekan pemain, dan HUD yang menampilkan skor, combo, serta timer. Screen ini menangani input penting seperti pause, serta memanggil subsistem lain seperti tile manager, score manager dari game_manager.

4.11.4 pause_screen

```

1 # Initialization
2 def __init__(self, width, height, game_manager=None): ...
3
4 # Event Handling
5 def handle_event(self, event): ...
6
7 # Button Navigation
8 def _next_button(self): ...
9 def _previous_button(self): ...
10 def _click_selected_button(self): ...
11
12 # Button Callbacks
13 def on_resume_clicked(self): ...
14 def on_settings_clicked(self): ...
15 def on_menu_clicked(self): ...
16 def _get_transition_for_button(self, button_name): ...
17
18 # Update Loop
19 def update(self): ...
20
21 # Rendering
22 def draw(self, surface): ...
23
24 # Lifecycle
25 def on_enter(self): ...
26 def on_exit(self): ...
27
28 # Status
29 def get_selected_button(self): ...
30 def __str__(self): ...

```

pause_screen berfungsi sebagai layar jeda yang muncul saat pemain menghentikan permainan sementara. Screen ini menampilkan overlay semi-transparan, tombol Resume, Settings, dan Menu, serta mendukung navigasi melalui keyboard dan mouse untuk memilih aksi lanjutan.

4.11.5 settings_screen

```

1 def __init__(self, width=1280, height=720, game_manager=None): ...
2 def _init_sliders(self): ...
3 def _init_difficulty_buttons(self): ...
4 def _init_song_selector(self): ...
5 def _init_buttons(self): ...
6 def _update_ui_from_settings(self): ...
7
8 # Event Handling
9 def handle_event(self, event): ...

```

```

10
11 # Update
12 def update(self): ...
13
14 # Drawing
15 def draw(self, surface): ...
16 def _draw_title(self, surface): ...
17 def _draw_difficulty_section(self, surface): ...
18 def _draw_song_section(self, surface): ...
19 def _draw_buttons(self, surface): ...
20
21 # Settings Application
22 def _apply_settings(self): ...
23 def get_settings(self): ...
24 def set_settings(self, settings): ...
25
26 # Lifecycle
27 def on_enter(self): ...
28 def on_exit(self): ...
29
30 # Status
31 def get_status(self): ...

```

Settings_screen adalah layar pengaturan yang memungkinkan pemain menyesuaikan preferensi permainan seperti tingkat kesulitan dan pilihan lagu, serta menampilkan UI interaktif berupa tombol, selector, dan elemen visual lainnya. Screen ini menangani update UI secara dinamis berdasarkan interaksi mouse dan menyimpan perubahan ke game_manager agar pengaturan langsung diterapkan.

5 Implementasi Program

5.1 Persiapan dan Clone Repository

Untuk memperoleh kode sumber aplikasi, pengguna perlu melakukan proses *clone* terhadap repository proyek. Langkah ini memastikan bahwa seluruh modul, aset audio, dan konfigurasi dapat diakses secara lengkap.

Langkah-langkah:

- Pastikan perangkat memiliki Git terpasang.
- Buka Terminal atau Command Prompt.
- Eksekusi perintah berikut:

```
1 git clone <URL-repository-AirBeats>
```

Masuk ke folder proyek:

```
1 cd AirBeats
```

5.2 Instalasi Dependencies

Aplikasi AirBeats memerlukan beberapa library Python seperti pygame, opencv-python, mediapipe, dan numpy untuk menjalankan fungsi multimedia, deteksi gesture, dan rendering permainan.

Langkah instalasi:

- Pastikan Python versi 3.8 atau lebih tinggi telah terpasang.
- Disarankan membuat virtual environment:

```
1 python -m venv venv
2 source venv/bin/activate      # Linux/Mac
3 venv\Scripts\activate         # Windows
```

Install dependencies menggunakan requirements.txt:

```
1 pip install -r requirements.txt
```

Jika file `requirements.txt` tidak tersedia, gunakan:

```
1 pip install pygame opencv-python mediapipe numpy
```

5.3 Menjalankan Program Utama

Aplikasi dijalankan melalui file `main.py`, yang memulai game loop, menyalakan kamera, memuat UI, dan mengaktifkan gesture detection.

Cara menjalankan:

- Pastikan kamera aktif dan tidak digunakan aplikasi lain.
- Jalankan perintah berikut:

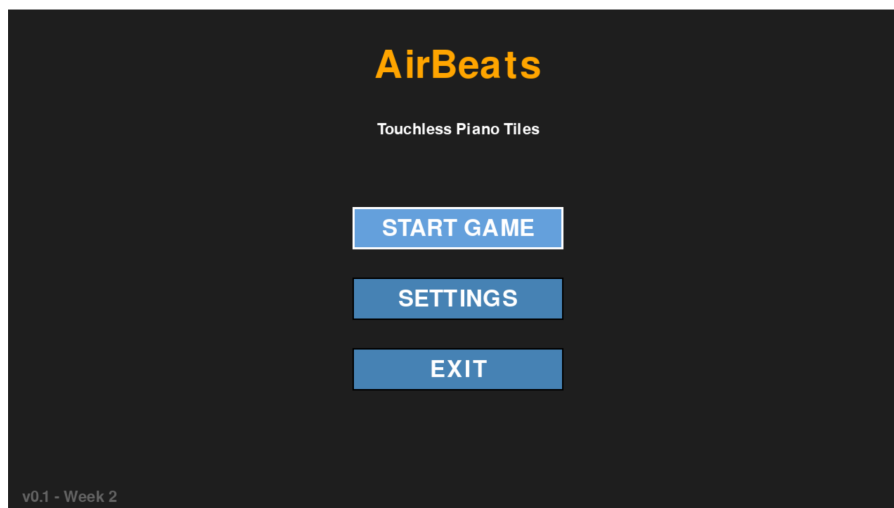
```
1 cd src
2 python main.py
```

Jendela permainan akan muncul menampilkan kamera serta antarmuka menu utama. Jika terjadi error kamera, pastikan webcam tidak sedang digunakan oleh aplikasi lain.

6 Penjelasan UI dan Mekanisme Interaksi

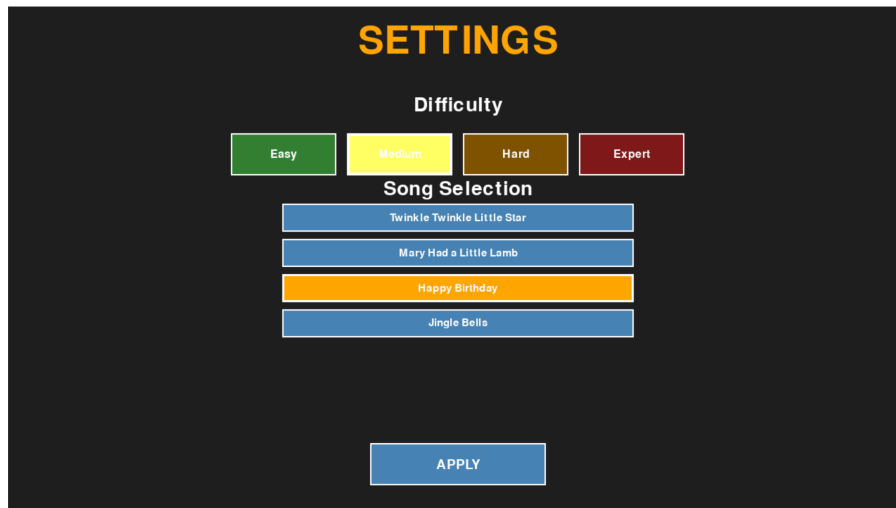
Aplikasi AirBeats menyediakan antarmuka yang intuitif berbasis Pygame serta interaksi gesture tanpa sentuhan menggunakan MediaPipe Hands.

Antarmuka Pengguna (UI):



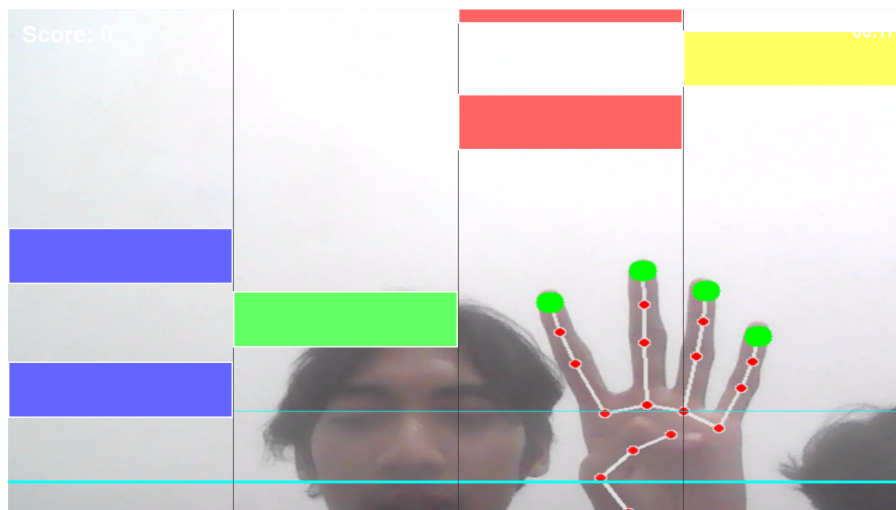
Gambar 1: Main Menu

- **Menu Utama** — Menampilkan tombol Play, Settings, dan Exit; navigasi menggunakan mouse.



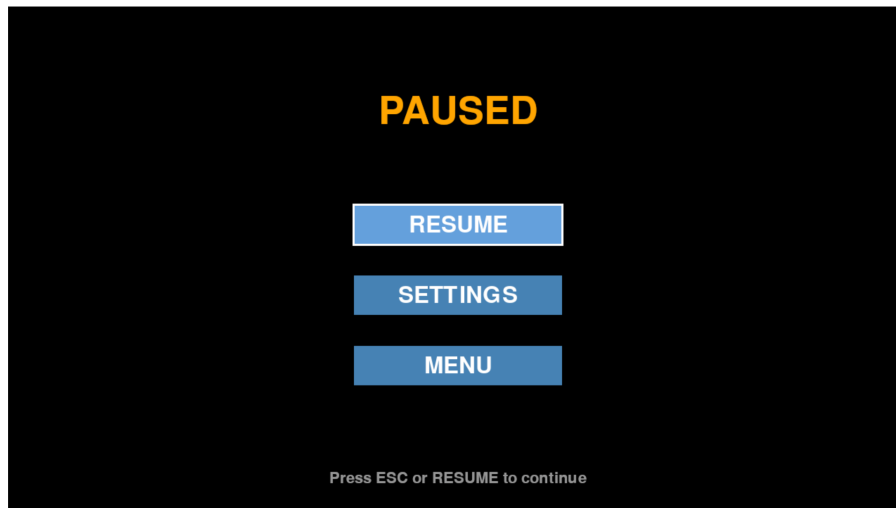
Gambar 2: Settings Menu

- **Settings Menu** — Menampilkan pengaturan permainan berupa pilihan lagu dan juga tingkat kesulitan bermain.



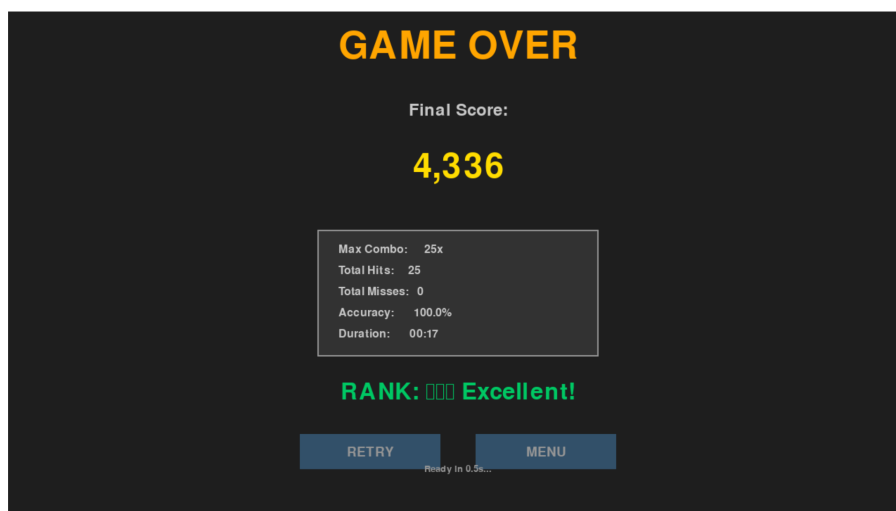
Gambar 3: Gameplay

- **Gameplay Screen** — Menampilkan kamera sebagai latar belakang, tile jatuh di empat lane, HUD skor, combo, timer, dan VFX.



Gambar 4: Pause Menu

- **Pause Screen** — Dibuka dengan tombol ESC; menyediakan opsi Resume dan kembali ke Menu.



Gambar 5: Gameover Screen

- **Game Over Screen** — Menampilkan skor akhir, akurasi, combo maksimum, dan peringkat performa.

Interaksi Gesture:

- Program melacak empat jari: index, middle, ring, dan pinky.
- Gerakan cepat ke bawah dianggap sebagai *tap*.
- Setiap jari mewakili satu lane:
 - Index → Lane 0
 - Middle → Lane 1
 - Ring → Lane 2
 - Pinky → Lane 3

- Ketepatan menentukan kualitas hit (PERFECT / GOOD / BAD).
- Audio nada piano diputar setiap kali gesture berhasil.

Aturan Interaksi Gameplay:

- Tile jatuh mengikuti timing lagu.
- Pemain harus melakukan tap saat tile memasuki zona hit.
- Miss menurunkan performa dan dapat menyebabkan game over.
- Efek visual muncul setiap kali hit terjadi.

7 Referensi dan Daftar Pustaka

References

- [1] Python Software Foundation, "Python programming language," <https://www.python.org/>, 2025.
- [2] G. AI, "Mediapipe hands documentation," https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2025.
- [3] OpenCV Team, "Opencv library," <https://opencv.org/>, 2025.
- [4] NumPy Developers, "Numpy: The fundamental package for scientific computing in python," <https://numpy.org/>, 2025.
- [5] Pygame Community, "Pygame: Python multimedia library," <https://pypi.org/project/pygame/>, 2025.

Referensi Penggunaan LLM

Dalam proses pengerjaan *project* ini, kami menggunakan beberapa bantuan penggunaan LLM yang dapat dilihat melalui link berikut:

- Bantuan Penyusunan Laporan: [ChatGPT](#)
- Bantuan Penyusunan Teknis Code 1: [Claude](#)
- Bantuan Penyusunan Teknis Code 2: [Antigravity Agent Manager](#)

Lampiran

- Link repository GitHub: <https://github.com/bintangfikrif/MM-Final-Project>
- Link Demo: <https://drive.google.com/drive/folders/lihKPCHknQireDc-MPuYUoWwTxccaUs2k>